



**Universidad
Tecnológica
del Perú**

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

S11.s2 — Trabajo Práctico

Curso

Inteligencia de Negocios

Sección

31677

Integrantes

Carrillo Barba, Alexis Martín	U21218567
Olivos Suxe, Neyser Alexander	U22315776
Pecho Santos, Manuel Angel	U76075762
Ramos Yampufe, Martin Alexander	U22214724
Neyra Nina, Bryan Smelin	U19204046

Docente

Tapia Ore, Willy Alexander

Perú

2026

Índice

Introducción	2
Arquitectura de Procesamiento y Enriquecimiento con Datos Abiertos	3
Hadoop vs. Spark	4
Arquitectura de Flujo	5
Flujo Propuesto	6
Enriquecimiento con Datos Abiertos	6
Pipeline Medallón	8
Detalle de cada capa	8
Estrategia NoSQL y Teorema de CAP	9
Justificación detallada	10
Conclusiones	11

Introducción

Olist, el marketplace brasileño objeto de estudio, enfrenta una alta tasa de retraso en las entregas que impacta negativamente en la satisfacción del cliente y genera pérdidas financieras estimadas en 2.5 millones de reales brasileños anuales. El presente trabajo práctico aborda el diseño detallado de la arquitectura de procesamiento Big Data para este problema, incluyendo la selección de tecnologías de cómputo, la definición del flujo de procesamiento, la estrategia de almacenamiento NoSQL gobernada por el Teorema de CAP y el enriquecimiento del análisis mediante fuentes de datos abiertos (Open Data).

Arquitectura de Procesamiento y Enriquecimiento con Datos Abiertos

Olist maneja un volumen estimado de 12.5 GB de datos por día (4.5 TB/año) con requerimientos de latencia inferiores a 5 segundos para alertas en tiempo real sobre retrasos en entregas. A continuación se define la infraestructura técnica para abordar este escenario.

Hadoop vs. Spark

Para el procesamiento de los datos de Olist se selecciona Apache Spark como motor principal de cómputo, sobre Apache Hadoop MapReduce, por las siguientes razones:

Tabla 1

Comparación entre Hadoop MapReduce y Apache Spark para el caso Olist

Criterio	Hadoop MapReduce	Apache Spark
Modelo de procesamiento	Batch únicamente	Batch, streaming, interactivo y ML
Velocidad	Lectura/escritura constante en disco	Procesamiento en memoria RAM (hasta 100x más rápido)
Latencia	Minutos a horas	Segundos a minutos
Facilidad para ML	Requiere integrar librerías externas	MLlib integrado (Random Forest, XGBoost)
Caso de uso en Olist	Serviría solo para el batch diario de entrenamiento	Cubre batch nocturno + streaming de alertas + dashboards interactivos

La elección de Spark se justifica porque Olist necesita tres modos de procesamiento simultáneamente:

1. Batch diario para reentrenar el modelo predictivo.
2. Micro-batches cada 5 minutos para actualizar dashboards operativos.
3. Procesamiento en tiempo real mediante Spark Streaming para detectar retrasos inminentes y emitir alertas.

Hadoop MapReduce solo cubre el primer escenario, lo que obligaría a mantener dos motores distintos y aumentaría la complejidad operativa.

No obstante, se propone conservar Hadoop HDFS como sistema de almacenamiento distribuido subyacente (data lake en crudo), aprovechando su bajo costo de almacenamiento en discos comerciales y su tolerancia a fallos mediante replicación. De esta forma, HDFS actúa como repositorio y Spark como motor de cómputo, combinando lo mejor de ambos ecosistemas.

Arquitectura de Flujo

Para el flujo de datos se selecciona la Arquitectura Lambda sobre la Arquitectura Kappa, fundamentado en los siguientes criterios:

Tabla 2

Comparación entre Arquitectura Lambda y Kappa para Olist

Criterio	Arquitectura Lambda	Arquitectura Kappa
Precisión histórica	Capa batch produce resultados exactos e inmutables	Depende de la reprocesabilidad del log inmutable de Kafka
Velocidad de respuesta	Capa speed ofrece latencia < 5s para alertas	Único motor streaming para todo
Complejidad de mantenimiento	Mayor (dos pipelines independientes)	Menor (un solo pipeline)
Idoneidad para Olist	Alta: requiere verdad absoluta en reportes financieros + alertas en tiempo real	Media: adecuada si se prioriza simplicidad sobre precisión histórica

La justificación de Lambda es la siguiente: Olist maneja transacciones comerciales con impacto financiero directo (reembolsos, devoluciones, multas por incumplimiento). Los reportes de retrasos consolidados que se presentan a los sellers deben ser exactos e inmutables (batch layer), mientras que las alertas tempranas requieren la mínima latencia posible (speed layer). Si bien Kappa simplifica la arquitectura al unificar ambos flujos en un solo motor de streaming, la tolerancia al error de Olist en los reportes históricos es cero; por lo tanto, la redundancia controlada de Lambda es una decisión de diseño justificada.

Flujo Propuesto

1. Ingesta mediante Apache Kafka que captura los eventos de órdenes, tracking y reseñas desde las fuentes transaccionales.
2. Procesamiento batch donde los datos crudos se almacenan en HDFS y cada noche Apache Spark procesa el lote diario, elimina duplicados, imputa valores faltantes y reentrena el modelo predictivo.
3. Procesamiento en tiempo real donde Spark Streaming consume el mismo tópico de Kafka, procesa ventanas de 5 segundos y detecta anomalías como un pedido con retraso proyectado mayor a 2 días, disparando alertas a los operadores logísticos.
4. Capa de servicio donde una base de datos NoSQL consolida los resultados de ambas capas y los expone al dashboard de Power BI o Tableau.

Enriquecimiento con Datos Abiertos

El pipeline descrito se beneficia de la incorporación de fuentes externas de datos abiertos (Open Data) que reducen la asimetría de información del modelo predictivo.

Variables como las condiciones climáticas, el tráfico urbano y los indicadores socioeconómicos regionales tienen un impacto directo en la logística de última milla y no están presentes en el dataset transaccional de Olist.

Se ha identificado el conjunto de datos abiertos *INMET — Dados Históricos Anuais* (Instituto Nacional de Meteorología de Brasil), disponible en el Portal Brasileiro de Dados Abertos (<https://dados.gov.br/dados/conjuntos-dados/inmet-dados-historicos>).

Este dataset contiene registros meteorológicos diarios de más de 500 estaciones de monitoreo distribuidas en todo Brasil, incluyendo las siguientes variables relevantes:

Tabla 3

Variables del dataset INMET y su relevancia para el análisis de retrasos en Olist

Variable	Descripción	Relevancia para Olist
Precipitación total (mm/día)	Cantidad de lluvia acumulada en 24 horas	Lluvias intensas retrasan rutas de transporte terrestre

Variable	Descripción	Relevancia para Olist
Temperatura máxima y mínima (°C)	Registro diario de temperatura ambiente	Temperaturas extremas afectan la manipulación de ciertos productos
Velocidad del viento (m/s)	Ráfagas máximas registradas	Vientos fuertes pueden cerrar rutas o demorar transportistas
Presión atmosférica (hPa)	Presión al nivel de la estación	Indicador indirecto de estabilidad climática
Ubicación geográfica	Latitud, longitud y altitud de la estación	Permite cruzar datos con ciudades de origen y destino de Olist

La incorporación de datos climáticos abiertos mejora la calidad del modelo predictivo de tres formas interrelacionadas:

1. Corrige la estacionalidad: sin datos climáticos, un retraso recurrente cada diciembre podría atribuirse erróneamente a un mal desempeño del transportista, cuando en realidad es consecuencia de la temporada de lluvias en el sudeste brasileño.
2. Permite una segmentación geográfica precisa: los promedios nacionales de retraso ocultan variaciones regionales significativas, y el cruce de coordenadas de las estaciones INMET con los códigos postales de clientes y vendedores revela qué rutas presentan mayor riesgo climático.
3. Alimentar el modelo Random Forest / XGBoost con la variable *precipitación acumulada en origen y destino* como feature adicional permite anticipar retrasos con mayor precisión, reduciendo el error absoluto medio (MAE) de la predicción.

Este enfoque demuestra cómo el Open Data actúa como un democratizador del mercado: una PYME como Olist puede acceder gratuitamente a inteligencia meteorológica que antes solo estaba al alcance de grandes corporaciones con presupuestos dedicados a la compra de datos privados.

Pipeline Medallón

El flujo de datos dentro del data lake se organiza siguiendo la Arquitectura Medallón, que garantiza la calidad progresiva de los datos a medida que avanzan desde su estado crudo hasta su consumo analítico:

Tabla 4

Arquitectura Medallón aplicada al pipeline de Olist

Capa	Función	Tecnología	Ejemplo en Olist
Bronze	Ingesta de datos crudos tal como llegan de la fuente, sin transformaciones	Apache Kafka + HDFS	JSON crudo de tracking: "order_id": "123", "lat": -23.5, "lng": -46.6, "timestamp": "2026-06-04T14:30:00Z"
Silver	Limpieza, filtrado, validación y normalización de datos. Eliminación de duplicados y estandarización de formatos	Apache Spark (ETL) + MongoDB (CP)	Tabla estructurada con ciudades normalizadas ("são paulo" → "São Paulo"), eliminación de eventos duplicados por error de red, imputación de fechas faltantes
Gold	Agregaciones finales, KPIs y datasets listos para consumo del negocio y modelos ML	Apache Spark + Cassandra (AP) + Power BI	KPI diario: "% de entregas a tiempo por categoría de producto y región"

Detalle de cada capa

- Capa Bronze: Los eventos generados por la aplicación web y los dispositivos de tracking GPS se publican en tópicos de Kafka. Un consumer en Spark Structured Streaming escribe los datos sin modificación en HDFS particionado por fecha. Esta capa funciona como un repositorio inmutable que permite reprocesar el histórico en caso de ser necesario. Adicionalmente, los datos meteorológicos del INMET se ingieren también en esta capa mediante jobs de carga programados diariamente, quedando disponibles en su formato original para su posterior transformación.

- Capa Silver: Un job ETL diario en Apache Spark lee los datos crudos de HDFS y aplica seis transformaciones secuenciales:
 1. Filtrado de registros con errores de formato o IDs inválidos.
 2. Deduplicación basada en *order_id + timestamp*.
 3. Normalización de nombres de ciudades mediante un diccionario geoespacial.
 4. Imputación de fechas de entrega faltantes usando la mediana de días por estado.
 5. Join con las tablas de clientes y vendedores para enriquecer la información geográfica.
 6. Cruce con los datos climáticos del INMET para agregar las variables meteorológicas por ciudad y fecha.

Los datos limpios y enriquecidos se persisten en MongoDB.

- Capa Gold: Un segundo job agregado calcula los KPIs estratégicos: tasa de retraso por categoría de producto, score promedio por vendedor, tiempo promedio de entrega por región y predicción de retraso del modelo ML enriquecida con variables climáticas. Estos resultados se almacenan en Cassandra para ser consumidos por el dashboard de Power BI con alta disponibilidad.

Estrategia NoSQL y Teorema de CAP

El diseño del almacenamiento NoSQL debe alinearse con los requisitos de cada capa del pipeline. El Teorema de CAP establece que, ante una partición de red, solo es posible garantizar dos de tres propiedades: consistencia (C), disponibilidad (A) o tolerancia a particiones (P). Dado que en sistemas distribuidos la partición de red es inevitable, la decisión real es entre CP (consistencia + tolerancia a particiones) y AP (disponibilidad + tolerancia a particiones).

Tabla 5

Estrategia NoSQL basada en el Teorema de CAP para Olist

Capa	Requisito	Elección NoSQL	Fundamento
Silver	Consistencia fuerte: los datos limpios deben ser precisos para reportes financieros	MongoDB (CP)	Si hay una partición de red, MongoDB prefiere bloquear la escritura antes que permitir datos inconsistentes. Esto es aceptable porque la capa Silver se actualiza por lotes (batch diario) y la consistencia es crítica para los cálculos de reembolsos
Gold	Disponibilidad alta: el dashboard del gerente debe responder siempre, aunque los datos tengan segundos de desfase	Apache Cassandra (AP)	Cassandra prioriza la disponibilidad: ante una partición, sigue sirviendo lecturas aunque algunos nodos no estén sincronizados. La consistencia eventual (milisegundos a segundos) es aceptable para KPIs analíticos y dashboards

Justificación detallada

- MongoDB (CP) en capa Silver: Las transacciones limpias en esta capa alimentan los reportes de retraso que determinan reembolsos a clientes y comisiones a vendedores. Un error de inconsistencia podría generar pagos incorrectos. MongoDB, al ser CP, sacrifica disponibilidad durante una partición de red —esto es aceptable porque el proceso ETL es batch y puede reintentarse minutos después sin afectar la operación en tiempo real.
- Cassandra (AP) en capa Gold: El dashboard de KPIs debe estar disponible para los gerentes de Olist en todo momento. Una caída de pocos segundos en la visualización podría retrasar decisiones comerciales críticas, como lanzar una promoción para una

categoría con alta tasa de retraso. Cassandra garantiza disponibilidad continua mediante consistencia eventual: si un nodo se desconecta momentáneamente, los demás continúan sirviendo datos y la sincronización ocurre de forma transparente en milisegundos.

Conclusiones

Apache Spark es la elección óptima como motor de procesamiento para Olist frente a Hadoop MapReduce, al cubrir simultáneamente los tres modos de procesamiento requeridos (batch, micro-batch y streaming) con velocidades hasta 100 veces superiores gracias al cómputo en memoria RAM, complementado por HDFS como sistema de almacenamiento distribuido de bajo costo. La Arquitectura Lambda es la más adecuada para el caso de negocio de Olist, ya que la necesidad de precisión absoluta en los reportes históricos de retrasos justifica la duplicación de pipelines frente a la alternativa Kappa, donde la capa batch garantiza la verdad inmutable mientras que la capa speed proporciona alertas en tiempo real. El uso de datos abiertos, específicamente el dataset meteorológico del INMET, permite reducir la asimetría de información en el modelo predictivo de Olist al incorporar variables climáticas que afectan directamente la logística de entregas, demostrando que el Open Data democratiza el acceso a inteligencia estratégica y nivela el campo de juego entre grandes corporaciones y PYMEs. La organización del pipeline bajo el modelo Medallón (Bronze, Silver, Gold) asegura una progresión controlada de la calidad del dato, desde la ingesta cruda en HDFS hasta los KPIs estratégicos consumidos desde el dashboard, con los datos abiertos integrándose desde la capa Bronze. Finalmente, la estrategia NoSQL dual, MongoDB (CP) para la capa Silver y Cassandra (AP) para la capa Gold, respeta las restricciones del Teorema de CAP alineando cada motor con los requisitos específicos de consistencia y disponibilidad de cada etapa del pipeline.